Oxide Systems Engineering

A mastery-gated course plan for whole-computer engineering

Primary path: Systems engineering at Oxide Computer Company

Core areas: Operating systems | Virtualization | Networking | Performance

Course philosophy: *Build. Instrument. Break. Debug. Explain.*

Course II: From low-latency systems engineer to whole-computer engineer.
Prerequisite: complete the Low-Latency Trading Engineering curriculum.

Contents

Mission and target candidate 3

How the curriculum works 4

Curriculum map 4

Stage 0 - Understand the computer you are targeting 4

Stage 1 - Rust systems mastery 5

Stage 2 - illumos, Helios, DTrace and MDB 6

Stage 3 - Kernel engineering and the hardware boundary 7

Stage 4 - Virtualization and VMM engineering 8

Stage 5 - Networking from wire to control plane 9

Stage 6 - Storage systems 10

Stage 7 - Control-plane and distributed-system engineering 11

Stage 8 - Embedded systems and hardware/software co-design 11

Stage 9 - Cross-stack performance and pathology 12

Stage 10 - Flagship project: Minirack 12

Stage 11 - Open-source engineering 14

Stage 12 - Build the Oxide application before applying 14

Portfolio structure 15

Areas of depth 15

Application gates 16

Asynchronous study loop 17

Failure-driven learning 17

Technical writing track 17

Reading Oxide effectively 17

Anti-rabbit-hole rules 18

Recommended resource stack 19

Definition of done 19

Selected references 20

Mission and target candidate

This asynchronous program builds a public body of systems work for Oxide Computer Company software engineering roles. Advance through mastery gates.

Assume the Low-Latency Trading Engineering curriculum has been completed in full.

You therefore already possess:

- strong modern C++.
- concurrency and memory-model knowledge.
- CPU/cache/memory knowledge.
- Linux systems competence.
- TCP/UDP and low-latency networking.
- performance profiling.
- latency measurement.
- distributed-systems experience.
- significant systems programming practice.
- a substantial trading-system project.
- experience reading serious systems code.
- algorithm/interview preparation.

This course does not repeat those subjects.

> *Primary goal: become an unusually compelling candidate for systems engineering at Oxide Computer Company, with enough breadth to credibly pursue operating-system, virtualization, networking, performance, and control-plane work - and enough public technical work to offset a relatively short professional career.*

The target is not merely: "I could probably do this job."

The target is: "This engineer is early in their career, but they have built, debugged, measured, documented, and contributed to an unusual amount of serious systems software."

What Oxide rewards

Oxide's current software roles point toward an unusually broad whole-system engineering profile. Kernel engineers work in C, Rust, and sometimes assembly and interact directly with processors and hardware. Virtualization engineers work across the illumos kernel, bhyve, Rust-based Propolis, virtual hardware, processors, and firmware. Networking engineers span Rust, C, P4, host networking, NICs, switches, drivers, BGP, overlays, IPsec, load balancing, and diagnosis. Performance engineers investigate from hardware through kernel and hypervisor into guests and applications, often creating new observability tooling. Control-plane engineers build distributed Rust systems connecting compute, storage, networking, telemetry, updates, and debugging infrastructure.

> *Central principle: learn to understand a computer vertically, not merely one software layer horizontally.*

The target candidate

At completion, your profile should approximately read:

> *Systems engineer experienced in high-performance software, Rust and C/C++, operating systems, virtualization, networking, distributed systems, and computer architecture. Has built an integrated miniature cloud-computing platform, implemented low-level OS/network/VMM components, conducted rigorous cross-stack performance investigations, and made meaningful contributions to production open-source systems projects. Strong technical writer comfortable documenting architecture, experiments, failures, and design decisions.*

Your distinguishing characteristic should become:

> *You build the thing, instrument the thing, break the thing, understand why it broke, fix it, and write down what you learned.*

How the curriculum works

The course is mastery-gated rather than time-gated. Follow the required path in the Curriculum map.

There are no calendar deadlines, but there is also no requirement that every topic be understood at research depth. Exit gates exist to prevent endless studying.

Curriculum map

Stage	Focus	Page
0	Oxide architecture and laboratory	4
1	Rust systems mastery	5
2	illumos, Helios, DTrace and MDB	6
3	Kernel engineering and the hardware boundary	7
4	Virtualization and VMM engineering	8
5	Networking from wire to control plane	9
6	Storage systems	10
7	Control-plane and distributed-system engineering	11
8	Embedded systems and hardware/software co-design	11
9	Cross-stack performance and pathology	12
10	Minirack flagship system	12
11	Open-source engineering	14
12	Oxide application portfolio	14

Stage 0 - Understand the computer you are targeting

Build a mental model of Oxide's system before attempting to specialize in it.

Core components

Component	Role
Helios	Host operating system built from illumos.
Propolis	Rust userspace VMM built around bhyve.
Omicron	Rust control plane.
Crucible	Distributed network-replicated block storage.
Hubris	Embedded Rust operating environment.
Dendrite / OPTE / networking stack	Rack and host networking components.

Required reading

- RFD 3 - Hiring Process.
- RFD 26 - Host Operating System & Hypervisor.
- RFD 60 - Storage Architecture.
- RFD 63 - Network Architecture.
- RFD 177 - Crucible implementation.

Repository reconnaissance

Clone and explore Helios, Propolis, Crucible, Omicron, and Hubris. Do not try to understand all of them. For each repository answer:

1. What problem does this component solve?
2. Where does it sit in the system?
3. What communicates with it?
4. What language is it written in?
5. What happens if it fails?
6. How is it tested?
7. How is it observed?
8. Where would I begin debugging it?

Deliverable: a concise architecture map of the Oxide software stack.

Stage 1 - Rust systems mastery

C++ is already one of your systems languages. Now Rust must become another. The goal is not to finish a book; it is to design and debug substantial systems software in Rust without translating C++ into Rust syntax.

Core Rust

- ownership and borrowing.
- lifetimes.
- enums and pattern matching.
- traits and generics.
- iterators and closures.
- Result and error propagation.
- modules and crates.

Memory and representation

- Box, Arc, Rc.
- Cell and RefCell.
- pinning.
- object representation and alignment.
- FFI.
- raw pointers.

Concurrency

- Send and Sync.
- atomics.
- mutexes.
- channels.
- scoped threads.
- async/await.
- futures.

Unsafe Rust

- raw pointer invariants.
- aliasing.
- UnsafeCell.
- uninitialized memory.
- FFI boundaries.
- unsafe traits.
- safe abstractions over unsafe implementations.

Projects

Rust systems port

Reimplement one nontrivial component from your C++ systems lab idiomatically in Rust. Document where Rust changes the architecture.

Binary protocol service

Implement a zero-copy-oriented binary protocol service. Investigate buffer ownership, parser APIs, error handling, and allocation behavior.

Concurrent subsystem

Implement a substantial concurrent component. Stress test it and use property-based testing, concurrency tests, and fault injection.

Stage 1 exit gate

- work comfortably in a large Cargo workspace.
- diagnose borrow/lifetime problems instead of randomly appeasing the compiler.
- design ownership intentionally.
- reason about Send and Sync.
- write asynchronous services.
- interface Rust with C.
- review unsafe Rust.
- profile Rust applications.
- explain where Rust safety guarantees stop.

Stage 2 - illumos, Helios, DTrace and MDB

Stop thinking only in Linux. Oxide's host operating system is Helios, built from illumos. This stage establishes serious operational and debugging fluency in that environment.

Build an illumos laboratory

Create a dedicated VM or physical system. Install an illumos-derived environment and eventually run Helios itself.

- processes and threads.
- zones.
- SMF.
- ZFS.
- device trees.
- /dev.
- illumos observability facilities.

Compare each with the Linux equivalents you already know.

DTrace

- probes and providers.
- predicates.
- aggregations.
- stack traces.
- syscall tracing.
- scheduler tracing.
- I/O tracing.
- network tracing.
- latency distributions.

Add your own instrumentation to a program. Your target skill is: something strange happened; you do not know why; you can instrument the running system until you know.

MDB

- process inspection.
- kernel inspection.
- stack examination.
- structures.
- postmortem analysis.
- crash dumps.

Stage project - Mystery Pathology

Create several intentional bugs, randomize which one is active, and diagnose the problem without looking at the implementation first.

- CPU saturation.
- lock contention.
- memory leak.
- blocked I/O.
- network latency.
- runaway syscall rate.

Deliverable: a postmortem for each pathology. These become early analysis-sample candidates.

Stage 3 - Kernel engineering and the hardware boundary

This stage transforms you from someone who understands an OS into someone who has modified one.

Operating-system implementation

- syscall implementation.
- virtual memory and page tables.
- scheduling.
- synchronization.
- kernel allocators.
- interrupts and timers.
- device discovery.
- driver architecture.
- kernel/user transitions.
- IPC.
- filesystems.
- security boundaries.

Read kernel source. Trace actual execution paths.

x86-64 / AMD64

Learn enough assembly to follow calling conventions, register use, stack frames, traps, interrupts, and context switching.

- page-table translation.
- TLBs.
- MSRs.
- APIC.
- privilege rings.
- exceptions.
- atomic instructions.
- memory ordering.

Use processor manuals selectively. The skill is learning to navigate imperfect hardware documentation when a real question appears.

Hardware interfaces

- PCIe.
- MMIO.
- DMA.
- IOMMU.
- device registers.
- interrupts / MSI-X.
- firmware.
- device discovery.
- NIC queues.
- NVMe architecture.

Kernel project

1. Implement a simple loadable module.
2. Build a character or pseudo device.
3. Expose kernel state through an appropriate interface.
4. Add statistics.
5. Add DTrace instrumentation.
6. Write MDB support or a debugging guide.

The artifact is source + tests + instrumentation + design document + debugging guide + failure analysis.

Stage 3 exit gate

You should be able to follow application -> syscall -> kernel -> driver -> device and explain where time, state, and failure can arise at each layer.

Stage 4 - Virtualization and VMM engineering

Concepts

- privileged execution.
- VM exits.
- hardware virtualization.
- guest/host transitions.
- nested page tables.
- virtual interrupts.
- vCPUs.
- device emulation.
- paravirtualization.
- virtio.
- IOMMUs.
- memory ballooning conceptually.
- VM lifecycle.
- live migration.
- time virtualization.

Practical progression

Start with KVM if it gives you the fastest experimentation environment. Then move to bhyve, Propolis, and Helios.

Virtual-device project

Implement a virtual device such as a simple virtio device, entropy device, virtual serial device, synthetic block device, or telemetry device.

- design specification.
- implementation.
- guest interaction.
- tracing.
- tests.
- failure behavior.
- benchmark.
- debugging guide.

Instrument the complete path: guest -> virtual device -> VMM -> host. Measure it. Explain every boundary crossing.

Advanced project

Modify or extend an existing open-source VMM. The contribution itself can be modest; the important evidence is that you can understand an unfamiliar VMM, trace a feature through it, change it safely, test it, and defend the design.

Stage 5 - Networking from wire to control plane

The trading curriculum already established sockets, TCP/UDP, multicast and performance networking. This stage starts both below and above that layer.

Host networking

- NIC descriptors.
- RX/TX rings.
- DMA.
- checksumming.
- TSO/GSO concepts.
- RSS.
- interrupt moderation.
- packet steering.
- hardware offloads.
- driver interaction.

Trace an incoming packet: wire -> NIC -> DMA -> driver -> kernel stack -> socket -> application, then reverse the path.

Routing and network architecture

- longest-prefix matching.
- ARP / NDP.
- IPv4 / IPv6.
- routing tables.
- ECMP.
- BGP.
- route advertisement.
- convergence.
- overlays.
- tunneling.
- NAT.
- firewalls.
- load balancing.
- VXLAN-style overlays.
- underlay vs overlay.
- virtual networking.
- east-west traffic.

IPsec

- ESP.
- security associations.
- key-management concepts.
- tunnel vs transport mode.
- replay protection.
- performance implications.

Software-defined networking

Study packet pipelines and learn basic P4. The goal is not mastery of switch silicon; it is comfort reasoning across host software, programmable datapaths, NICs, and switching hardware.

Networking flagship subsystem

Build a Rust network dataplane with:

- L2/L3 forwarding.
- routing table.
- ACL/firewall rules.
- NAT.
- simple overlay.
- metrics.
- packet tracing.

Add a small control plane. Eventually add dynamic routing - preferably BGP - plus failure injection and convergence measurements. Make it correct first; then profile the forwarding path.

Networking analysis sample

Create and investigate a pathological scenario such as asymmetric routing, route flap, MTU mismatch, packet loss, head-of-line blocking, bad offload interaction, or reordering. Produce a rigorous root-cause report.

Stage 6 - Storage systems

Even if no storage-specific opening exists when you apply, storage is central to the machine.

Foundations

- block devices.
- sectors and alignment.
- NVMe.
- queues.
- fsync / flush semantics.
- write ordering.
- durability.
- checksums.
- crash consistency.
- snapshots.
- copy-on-write.

ZFS

- pools.
- vdevs.
- ARC.
- transaction groups.
- checksums.
- copy-on-write.
- snapshots.

Distributed storage

- replication.
- quorum concepts.
- ordering.
- degraded operation.
- repair.
- reconciliation.
- split brain.
- failure detection.
- durability vs availability.
- consistency.

Storage project

Build a miniature replicated block device in Rust: client -> three replicas. Support read, write, and flush. Then add replica failure, restart, repair, checksums, corruption detection, metrics, and tracing.

Failure laboratory

Test process crash during write, replica timeout, stale replica, corrupted block, partial write, and network partition. For each:

1. Predict behavior.
2. Run the failure.
3. Observe behavior.
4. Determine whether your prediction was correct.
5. Fix incorrect assumptions.

Stage 7 - Control-plane and distributed-system engineering

This stage builds directly on your backend/distributed-systems experience and should therefore be relatively focused.

Focus areas

- reconciliation.
- desired vs observed state.
- state machines.
- sagas/workflows.
- idempotency.
- rolling software update.
- schema evolution.
- partial failure.
- distributed debugging.
- API evolution.
- resource allocation.
- inventory.
- authentication/attestation concepts.

Property-based testing

Use property-based testing aggressively, particularly for state machines, resource lifecycle, network configuration, and storage operations. Generate adversarial event sequences.

Control-plane project

Build a Rust controller for the systems developed elsewhere in the course. Maintain desired state for compute instances, virtual networks, and storage volumes. Reconcile actual state toward desired state.

Then deliberately kill agents, restart controllers, reorder events, replay messages, and disconnect nodes. Prove that the system converges.

Stage 8 - Embedded systems and hardware/software co-design

You do not need to become an electrical engineer. You do need to understand that the computer continues below the host CPU.

Embedded foundations

- microcontrollers.
- memory-mapped peripherals.
- interrupts.
- GPIO.
- SPI.
- I2C.
- UART.
- timers.
- watchdogs.
- firmware updates.
- bootloaders.

Use Rust.

Hardware lab

Purchase an inexpensive supported development board. Implement a sensor or peripheral driver, asynchronous event handling, watchdog, telemetry, and fault recovery.

Then deliberately violate assumptions: peripheral disappears, invalid value appears, bus times out. Design recovery.

Firmware artifact

Produce one small embedded project of unusually high quality, including reference documentation, Rust firmware, protocol description, host-side tooling, fault-handling strategy, test strategy, and analysis.

Stage 9 - Cross-stack performance and pathology

The quant curriculum gave you a performance-engineering advantage. This stage turns that into cross-stack diagnosis.

Performance investigations

Layer	Example concerns
CPU	cache misses, branch behavior, contention, instruction count
Kernel	scheduling, interrupts, system calls, memory pressure
Virtualization	VM exits, virtual I/O, scheduling
Networking	packet path, drops, queueing, offload behavior
Storage	queue depth, flush latency, replication latency
Distributed system	request fan-out, tail latency, retry storms, lockstep behavior

Rule

> *Do not merely create benchmarks. Create mysteries.*

Example: p99 VM disk latency unexpectedly increases 8x under a particular network load. Investigate until you know why.

A high-quality report contains symptom, initial hypotheses, instrumentation, measurements, eliminated hypotheses, root cause, fix, validation, and remaining uncertainty.

Tool-building requirement

Encounter at least one problem where existing tooling is inadequate, then build the missing tool - for example a DTrace script, Rust diagnostic utility, packet analyzer, visualization helper, profiler helper, or crash-inspection utility.

Stage 10 - Flagship project: Minirack

This becomes the centerpiece of your Oxide portfolio. Do not build an "Oxide clone." Build a small system inspired by the same category of engineering problems.

> *Minirack: a miniature open-source cloud-computer laboratory.*

Architecture

At completion: Rust API/control plane -> node agent -> VM manager -> virtual networking -> replicated block storage, with telemetry, failure injection, reconciliation, and debug tooling.

Compute

- manage VM lifecycle.
- create.
- start.
- stop.
- destroy.
- query.

Initially use an existing hypervisor. Later integrate your virtualization work where appropriate.

Storage

- create volume.
- attach.
- detach.
- failure/recovery.

Networking

- isolated virtual networks.
- addressing.
- routing.
- firewalling.
- overlay networking.

Control plane

Expose a clean API. Maintain desired state versus observed state. Recover from agent restarts, stale state, partial operations, and node loss.

Observability

Every subsystem should expose structured logs, metrics, and traces, plus diagnostic commands.

Failure injection

- kill storage replica.
- kill VM agent.
- drop packets.
- delay packets.
- corrupt a disk block.
- crash during resource creation.

Your system should fail visibly rather than mysteriously.

Upgrade experiment

Implement at least one rolling software update where different component versions temporarily coexist. Document compatibility strategy.

Physical deployment

Eventually run Minirack on multiple physical machines. Three inexpensive x86 machines are enough. Real hardware exposes NIC behavior, boot issues, storage behavior, firmware, topology, and failure modes that simulations hide.

Minirack documentation

- architecture - what exists and why.
- failure model - what can fail.
- consistency model - what guarantees exist.
- security boundaries - who trusts whom.
- update model - how software changes.
- performance model - expected bottlenecks.
- debugging guide - how failures are investigated.
- RFD history - major architectural decisions.

The repository should sustain an hour of serious technical discussion with an Oxide engineer.

Stage 11 - Open-source engineering

Open-source work is mandatory, but do not contribute performatively to Oxide repositories merely to get noticed. Respect maintainer capacity and project guidance.

Contribution targets

- illumos.
- rust-vmm.
- Rust systems libraries.
- networking tools.
- DTrace ecosystem.
- virtualization tools.
- storage tools.
- selected Oxide projects where contribution guidance permits.

Contribution progression

1. Small but real defect; include a test.
2. More substantial bug or feature; participate actively in review.
3. Performance, concurrency, OS or protocol work; write an accompanying technical explanation.

The target is evidence that experienced systems maintainers trusted your reasoning enough to merge your work.

Upstreaming mindset

- identify maintainer priorities.
- reproduce bugs rigorously.
- reduce test cases.
- write useful commit messages.
- accept review.
- revise designs.
- avoid unnecessary cleverness.
- support your changes.

Stage 12 - Build the Oxide application before applying

For Oxide, application material is a central engineering artifact. This course intentionally produces work samples, writing samples, and analysis samples rather than treating them as an afterthought.

Work samples

Work Sample A - Minirack

Shows Rust, distributed systems, virtualization, networking, storage, integration, failure handling, and architecture.

Work Sample B - Deep low-level subsystem

For example: kernel component, VMM feature, networking dataplane, or driver. Shows genuine depth.

Work Sample C - Open-source contribution

Shows unfamiliar-codebase navigation, collaboration, code review, and production constraints.

Analysis samples

- cross-stack performance pathology.
- kernel or VMM failure investigation.
- networking pathology or storage corruption/recovery analysis.

These should read like engineering investigations, not school reports.

Writing sample

Produce one serious RFD-style document. Good topics include live migration for Minirack, network configuration reconciliation, or crash-consistent replicated volume metadata.

- problem definition.
- constraints.
- alternatives.
- tradeoffs.
- decision.
- unresolved questions.

Portfolio structure

Do not create 40 toy repositories. Aim for:

Artifact	Purpose
minirack	Integrated flagship system.
systems-lab	Kernel, virtualization, embedded and diagnostic experiments.
one deep subsystem	Networking dataplane, VMM work, or kernel project.
open-source work	External contributions with review history.
engineering notebook/site	Polished technical investigations and RFD-style writing.

Areas of depth

Operating systems / virtualization

You must have modified real kernel or VMM-level software - not merely read about it.

Networking

You must understand both packet mechanics and network architecture: NIC/driver concepts through BGP/overlay systems.

Performance

You must have conducted several investigations where the answer was not known before measurement.

These three become your signature. Storage, embedded and control-plane knowledge ensure breadth across the rest of the stack.

Application gates

First application gate

Do not apply simply because you have read all the material. For Oxide, wait until the evidence is unusually strong.

Area	Minimum application threshold
Rust	Ship and debug significant Rust systems code.
illumos	Comfortable working in an illumos environment.
DTrace/MDB	Use them for real investigations.
Kernel	Have modified kernel-level software.
Hardware	Reason about PCIe, MMIO, DMA, interrupts, and device behavior.
Virtualization	Understand hardware virtualization and have modified or implemented VMM/device-emulation code.
Networking	Understand networking from NICs through BGP/overlays and have built substantial networking software.
Storage	Understand block storage and distributed replication.
Control plane	Design reconciliation and failure recovery.
Performance	At least three serious measured investigations.
Minirack	A substantial portion works.
Open source	Meaningful third-party-reviewed systems work.
Writing	At least one excellent design document.
Analysis	At least one excellent technical investigation suitable for submission.
Oxide knowledge	Intelligently discuss Helios, Propolis, Omicron, Crucible, Hubris, networking architecture, and architectural choices.

At this point: apply. Do not wait until you become a kernel veteran; that is not an objective gate.

Exceptional-candidate gate

- multiple nontrivial open-source contributions.
- deep Rust competence.
- substantial real illumos experience.
- a functioning kernel or driver artifact.
- a meaningful virtualization artifact.
- substantial networking software.
- three-node Minirack deployment.
- rigorous failure testing.
- custom diagnostic tooling.
- multiple performance analyses.
- RFD-quality writing.
- repository history showing months of disciplined engineering.
- ability to explain failures as clearly as successes.

At this point your narrative becomes: "I deliberately spent years becoming a whole-system engineer, and here is the evidence."

Asynchronous study loop

Step	Action
Study	Learn enough theory to form a model.
Close the material	Remove the reference.
Reconstruct from memory	Explain the system yourself.
Implement	Turn the model into code.
Instrument	Make behavior observable.
Break	Create failure intentionally.
Debug	Find the real cause.
Explain	Write down the reasoning.
Revisit later	Recover the model after time has passed.

> *The most powerful retrieval question in systems work is often: "What do I predict the machine will do?" Make the prediction, run the experiment, then explain why you were wrong.*

Failure-driven learning

Most students build projects whose happy path works. This course should deliberately build systems whose unhappy paths are understood.

- process crash.
- power-loss approximation.
- packet loss.
- corruption.
- latency.
- partial state.
- version mismatch.
- stale data.
- overload.
- concurrent operations.

Senior systems engineers possess enormous stores of failure experience. You cannot reproduce their years, but you can deliberately accumulate far more failure experience than the average engineer at your career stage.

Technical writing track

Writing is not a final polish step. Every major system should produce:

- design note - before implementation.
- decision record - when architecture changes.
- debugging note - when something surprising happens.
- postmortem - after a serious failure.
- performance analysis - after optimization.
- final architecture document - when complete.

Over time, delete weak writing and polish the best work. Your archive becomes application inventory.

Reading Oxide effectively

Do not read every RFD, listen to every podcast episode, or memorize company trivia. Use public material when you reach the relevant technical area.

When studying...	Use...
Virtualization	RFD 26 and Propolis material
Storage	RFD 60 and RFD 177
Networking	RFD 63
Performance mysteries	Relevant Oxide performance investigations

The goal is to become an engineer comparing architectural decisions with your own experience - not a fan studying a company.

Anti-rabbit-hole rules

1. Never implement an entire technology when modifying a real implementation teaches the same lesson.
2. A textbook chapter should usually lead to source code within days, not months.
3. Prefer debugging a real system over building another toy.
4. One excellent failure analysis is worth more than ten shallow projects.
5. Do not optimize before measurement.
6. Do not contribute to open source merely for resume optics.
7. Do not wait until Minirack is "finished." Infrastructure software is never finished.
8. Do not compare your breadth directly with a twenty-year kernel engineer; compare the quality of evidence you can produce today.
9. If a topic does not improve your code, analysis, writing, debugging ability, or application evidence, defer it.

Things deliberately not required

Do not delay applying because you have not:

- written a complete kernel.
- written a complete hypervisor.
- designed a CPU.
- become an electrical engineer.
- designed a motherboard.
- mastered every network protocol.
- implemented BGP from scratch.
- implemented a production filesystem.
- learned all of illumos.
- learned every Rust feature.
- read every Oxide RFD.
- contributed directly to Oxide.
- built an actual rack-scale computer.

These are rabbit holes. The goal is credible direct experience with the important problems.

Recommended resource stack

Area	Core resources
Rust	Programming Rust; official Rust docs; Rustonomicon when required; existing Rust systems code.
Operating systems	Operating Systems: Three Easy Pieces; selected CS:APP material; illumos source/docs; Helios source.
Observability	DTrace docs; MDB docs; Brendan Gregg systems- performance material.
Hardware	AMD/Intel architecture manuals as references; PCIe/NVMe docs when projects require them.
Virtualization	bhyve; Propolis; rust-vmm ecosystem; processor virtualization docs.
Networking	TCP/IP Illustrated selectively; relevant RFCs; real implementations; Oxide RFD 63.
Storage	ZFS docs; Oxide RFD 60; RFD 177; Crucible source.
Embedded	Embedded Rust material; Hubris docs/source.
Distributed systems	Use Omicron architecture as the primary learning object rather than restarting a generic textbook.

Definition of done

The ultimate output of this course is not knowledge. It is a body of evidence.

Claim	Evidence
I understand Rust.	Here is the code.
I understand operating systems.	Here is the kernel work.
I understand virtualization.	Here is the VMM work.
I understand networking.	Here is the dataplane and its failure analysis.
I understand storage.	Here is the replicated block system.
I understand distributed systems.	Here is Minirack's control plane.
I understand hardware.	Here is the driver/embedded work.
I understand performance.	Here are the investigations.
I can debug.	Here are the postmortems.
I can communicate.	Here are the design documents.
I can work with other engineers.	Here are the upstream contributions and review discussions.

You cannot manufacture someone else's years. You can build an unusually dense body of serious engineering work. That is what this curriculum is designed to do.

Selected references

Careers: https://oxide.computer/careers
Performance Engineer: https://oxide.computer/careers/performance-engineer
OS Kernel Engineer: https://oxide.computer/careers/os-kernel
Virtualization Engineer: https://oxide.computer/careers/sw-host-virt
Networking Software Engineer: https://oxide.computer/careers/sw-networking
Control Plane Engineer: https://oxide.computer/careers/sw-control-plane
RFD 3 - Hiring Process: https://rfd.shared.oxide.computer/rfd/0003
RFD 26 - Host OS & Hypervisor: https://rfd.shared.oxide.computer/rfd/0026
RFD 60 - Storage Architecture: https://rfd.shared.oxide.computer/rfd/0060
RFD 63 - Network Architecture: https://rfd.shared.oxide.computer/rfd/0063
RFD 177 - Crucible: https://rfd.shared.oxide.computer/rfd/0177
Helios: https://github.com/oxidecomputer/helios
Crucible: https://github.com/oxidecomputer/crucible
Omicron: https://github.com/oxidecomputer/omicron
Hubris: https://github.com/oxidecomputer/hubris